



CSC 316 – COMPILER CONSTRUCTION I

Week Four:

Parsing (Syntax Analysis)

Recommended Texts for this Course

1. **Introduction to Compilers and Language Design** Copyright © 2023 Douglas Thain.
Paperback ISBN: 979-8-655-18026-0 Second edition.
2. Campbell, Bill; Iyer, Swami; and Akbal-Delibas, Bahar (2013). **Introduction to Compiler Construction in a Java World**, Taylor & Francis Group

Parsing

- Once we have identified the tokens in our program, we then want to determine its syntactic structure.
- That is, we want to put the tokens together to make the larger syntactic entities: expressions, statements, methods, and class definitions.
- This process of determining the syntactic structure of a program is called *parsing*.

- First, we wish to make sure the program is syntactically valid - that it conforms to the Grammar that describes its syntax.
- As the Parser parses the program, it should identify syntax errors and report them and the line numbers they appear on.
- Moreover, when the Parser does find a syntax error, it should not just stop, but it should report the error and gracefully recover so that it may go on looking for additional errors.

- Second, the parser should produce some representation of the parsed program that is suitable for semantic analysis.
- A BNF (**Backus–Naur Form (BNF)**) grammar describes the syntax of programs in a programming language.
- The Abstract Syntax Tree, (AST) captures the essential syntax of our program.
- Why are we interested in a Tree representation for our program
 - ✓ Because it is easier to analyze and decorate (with type information) a Tree than it is to do the same with text.
- The AST makes that syntax, which is implicit in the program text, explicit.
 - ✓ This is the purpose of Parsing.

Context-Free Grammars and Languages

Backus–Naur Form (BNF) and Its Extensions

- The grammars that we use to describe programming languages are inherently recursive and so are best described by what we call context-free grammars.
- For example, the context-free rule
 - $S ::= \text{if}(E)S$
 - says that, if E is an expression and S is a statement,
 - $\text{if}(E)S$ (1)
 - is also a statement.
 - Or, we can read the rule as, a statement S can be written as an if, followed by a parenthesized expression E and finally a statement S .
 - That is, we can read the “ $::=$ ” as ‘can be written as a’.

- There are abbreviations possible in the notation.
- For example, we can write

$$S ::= \text{if}(E) S \mid \text{if}(E) S \text{ else } S \quad (2)$$

as shorthand for

$$\begin{aligned} S &::= \text{if}(E) S \\ S &::= \text{if}(E) S \text{ else } S \end{aligned} \quad (3)$$

That is, we can read the ‘|’ as ‘**or**’.

- So, a statement **S** can be written either as an if followed by a parenthesized expression **E**, and a statement **S**;
- or as an **if**, followed by a parenthesized expression **E**, a statement **S**, an else, and another statement **S**.

- In our grammar, the square brackets indicate that a phrase is optional..
- For example, (2) and (3) could have been written as

$$S ::= \text{if } (E) S \text{ [else } S] \quad (4)$$

which says an S may be written as $\text{if } (E) S$, optionally followed by $\text{else } S$.

- Curly braces denote the Kleene closure, indicating that the phrase may appear zero or more times
- For example,

$$E ::= T \{ + T \} \quad (5)$$

which says that an expression E may be written as a term T , followed by zero or more occurrences of $+$ followed by a term T :

T

$T + T$

$T + T + T$

• ...

- Finally, one may use the alternation sign $|$ inside right-hand sides, using parentheses for grouping.
- For example,

$$E ::= T \{ (+ \mid -) T \}$$

meaning that the additive operator may be either $+$ or $-$, allowing for:

$$T + T - T + T$$

Grammar and the Language It Describes

- A context-free grammar is a tuple, $G = (N, T, S, P)$, where
 - N is a set of non-terminal symbols, sometimes called non-terminals;
 - T is a set of terminal symbols, sometimes called terminals;
 - $S \in N$ is a designated non-terminal, called the start symbol,; and
 - P is a set of production rules, sometimes called productions or rules.
- For example, a context-free grammar that describes (very) simple arithmetic expressions is $G = (N, T, S, P)$.
- where $N = \{E, T, F\}$ is the set of non-terminals, $T = \{+, *, (,), \text{id}\}$ is the set of terminals, $S = E$ is the start symbol, and the Production set, P is as follows:
- $P = \{E ::= E + T, E ::= T, T ::= T * F, T ::= F, F ::= (E), F ::= \text{id}\}$

- A little less formally, we can denote the same grammar simply as a sequence of production rules.
- For example, the grammar below denotes the same grammar as does the immediate preceding one given.

$E ::= E + T$

$E ::= T$

$T ::= T * F$

$T ::= F$

$F ::= (E)$

$F ::= \text{id}$

- We may surmise that the symbols (here E , T , and F) that are defined by at least one production rule are non-terminals.
- Conventionally, the first defined non-terminal (that is, E here) is our start symbol.
- Those symbols that are not defined (here $+$, $*$, $($, $)$, and id) are terminals.
- The start symbol is important to us because it is from this symbol, using the production rules, that can generate strings in a language.
- For example, because we designate E to be the start symbol in the grammar above, we can record a sequence of applications of the production rules, starting from E to the sentence **id + id * id** as follows:

$E \Rightarrow E + T$
 $\Rightarrow T + T$
 $\Rightarrow F + T$
 $\Rightarrow \text{id} + T$
 $\Rightarrow \text{id} + T * F$
 $\Rightarrow \text{id} + F * F$
 $\Rightarrow \text{id} + \text{id} * F$
 $\Rightarrow \text{id} + \text{id} * \text{id}$

...We call this a *derivation*.

- When a string of symbols derives another string of symbols in one step, that is, by a single application of a production rule, we say that **first string *directly derives* the second string**.
- For example,
 - E directly derives
 - $E + T$
 - $E + T$ directly derives
 - $T + T$
 - $T + T$ directly derives
 - $F + T$. and so on ...
- When one string can be rewritten as another string, using zero or more production rules from the grammar, we say the first string *derives* the second string.
- For this *derives* relation, we usually use the symbol $\Rightarrow$.
- For example,
 - $E \Rightarrow * E$ (in zero steps)
 - $E \Rightarrow * id + F * F$
 - $T + T \Rightarrow * id + id * id$

- We say the *language* $L(\mathbf{G})$ that is described by a grammar $\mathbf{G}$ consists of all the strings (sentences) comprised of only terminal symbols that can be derived from the start symbol.
- A little more formally, we can express the language $L(\mathbf{G})$ for a grammar $\mathbf{G}$ with start symbol $\mathbf{S}$ and terminals $\mathbf{T}$ as
- $L(\mathbf{G}) = \{w \mid \mathbf{S} \Rightarrow^* w \text{ and } w \in \mathbf{T}^*\}.$
- For example, in our grammar above,

$$E \Rightarrow * \text{ id } + \text{ id } * \text{ id }$$

$$E \Rightarrow * \text{ id }$$

$$E \Rightarrow * (\text{ id } + \text{ id }) * \text{ id }$$

so, $L(\mathbf{G})$ includes each of

id

(id + id) * id

- and infinitely more finite sentences.

Top-Down and Bottom-Up parsing Approaches

- There are two major parsing approaches: *top-down* and *bottom-up*.
- In **top-down** parsing, you start with the **start symbol**, **S** and apply the **production rules** until you arrive at the **desired/specified string**.
- In **bottom-up** parsing, you start with the **specified string** and reduce it to the **start symbol**, **S** by applying the **production rules** backwards.
- As an example, let's trace through the two approaches on this simple grammar that recognizes strings consisting of any number of **a's** followed by at least one (and possibly more) **b's**:

- **$S \rightarrow AB$**
- **$A \rightarrow aA \mid \epsilon$**
- **$B \rightarrow b \mid bB$**

Here is a top-down parse of aaab. >>>

top-down parse

- We begin with the start symbol and at each step, expand one of the remaining nonterminals by replacing it with the right side of one of its productions.
- We repeat until only terminals remain.
- The top-down parse produces a leftmost derivation of the sentence.

Start Symbol, S

AB

aAB

aaAB

aaaAB

aaaεB

aaab

Rules:

$S \rightarrow AB$

$A \rightarrow aA$

$A \rightarrow aA$

$A \rightarrow aA$

$A \rightarrow \epsilon$

$B \rightarrow b$

bottom-up parse

- A bottom-up parse works in reverse.
- We begin with the sentence of terminals and each step applies a production in reverse, replacing a substring that matches the right side with the nonterminal on the left.
- We continue until we have substituted our way back to the start symbol.

The string

- **aaab**
- **aaaεb**
- **aaaAb**
- **aaAb**
- **aAb**
- **Ab**
- **AB**
- **S**

The Rules:

(insert ε)

$A \rightarrow \varepsilon$

$A \rightarrow aA$

$A \rightarrow aA$

$A \rightarrow aA$

$B \rightarrow b$

$S \rightarrow AB$

Derivations (cont'd)

- We are interested in languages, those **strings of terminals** that can be derived from a grammar's **start symbol**.
- There are two kinds of derivation that will be important to us when we go about parsing these languages: **left-most derivations** and **right-most derivations**.
- A **left-most derivation** is a derivation in which at each step, the next string is derived by applying a production rule for rewriting the *left-most* non-terminal.
- For example, we have already seen a left-most derivation of $\text{id} + \text{id} * \text{id}$:

$$\begin{aligned}\underline{E} &\Rightarrow \underline{E} + T \\ &\Rightarrow \underline{T} + T \\ &\Rightarrow \underline{F} + T \\ &\Rightarrow \text{id} + \underline{T} \\ &\Rightarrow \text{id} + \underline{T} * F \\ &\Rightarrow \text{id} + \underline{F} * F \\ &\Rightarrow \text{id} + \text{id} * \underline{F} \\ &\Rightarrow \text{id} + \text{id} * \text{id}\end{aligned}$$

- Here we have underlined the left-most non-terminal in each string of symbols in the derivation to show that it is indeed a left-most derivation.

right-most derivation

- A **right-most derivation** is a derivation in which at each step, the next string is derived by applying a production rule for rewriting the *right-most* non-terminal.
- For example, the right-most derivation of $\text{id} + \text{id} * \text{id}$ would go as follows:

$$\begin{aligned} \underline{E} &\Rightarrow E + \underline{T} \\ &\Rightarrow E + T * \underline{E} \\ &\Rightarrow E + \underline{T} * \text{id} \\ &\Rightarrow E + \underline{E} * \text{id} \\ &\Rightarrow \underline{E} + \text{id} * \text{id} \\ &\Rightarrow \underline{T} + \text{id} * \text{id} \\ &\Rightarrow \underline{E} + \text{id} * \text{id} \\ &\Rightarrow \text{id} + \text{id} * \text{id} \end{aligned}$$

- We use the term *sentential form* to refer to any string of (terminal and non-terminal) symbols that can be derived from the start symbol.
- So, for example in the previous derivation,

E

$E + T$

$E + T * F$

$E + T * \text{id}$

$E + F * \text{id}$

$E + \text{id} * \text{id}$

$T + \text{id} * \text{id}$

$F + \text{id} * \text{id}$

$\text{id} + \text{id} * \text{id}$

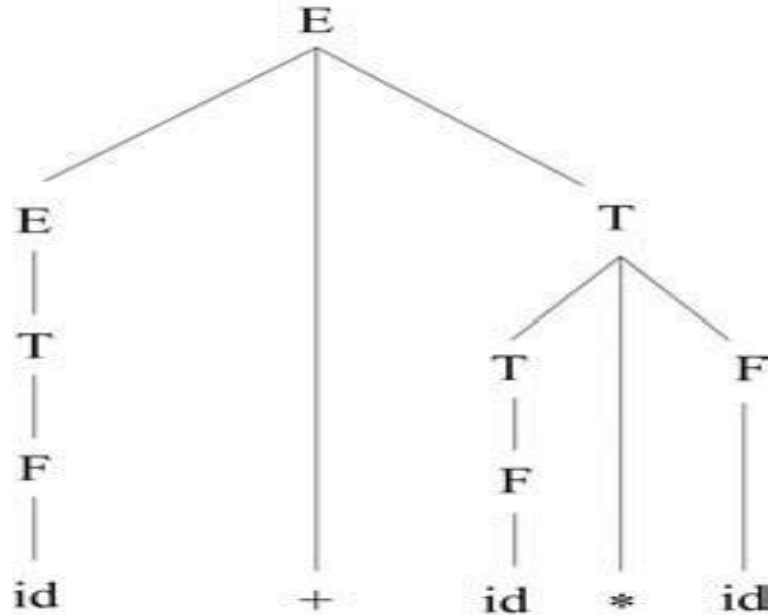
are all sentential forms.

➤ Clearly, any sentential form consisting solely of terminal symbols is a sentence in the language.

Parse tree

- An alternative representation of a derivation is the *parse tree*, a tree that illustrates the derivation and the structure of an input string (at the leaves) from a start symbol (at the root).
- For example, Figure 1 shows the parse tree for $\text{id} + \text{id} * \text{id}$.

A parse tree for **id + id * id**



Ambiguous Grammars and Unambiguous Grammars

- Given a grammar G , if there exists a sentence s in $L(G)$ for which there are more than one left-most derivations in G (or equivalently, either more than one right-most derivations, or more than one parse tree for s in G), we say that the sentence s is *ambiguous*.
- Moreover, if a grammar G describes at least one ambiguous sentence, the grammar G is *ambiguous (grammar)*.
- If there is no such sentence, that is, if every sentence is derivable by a unique left-most (or right-most) derivation (or has a unique parse tree), we say the grammar is *unambiguous*.

Example

- Consider the grammar, G:

$$E ::= E + E \mid E * E \mid (E) \mid \text{id}$$

and, consider the sentence form, $\text{id} + \text{id} * \text{id}$.

- One left-most derivation for this sentence is:

$$\begin{aligned} \underline{E} &\Rightarrow \underline{E} + E \\ &\Rightarrow \text{id} + \underline{E} \\ &\Rightarrow \text{id} + \underline{E} * E \\ &\Rightarrow \text{id} + \text{id} * \underline{E} \\ &\Rightarrow \text{id} + \text{id} * \text{id} \end{aligned}$$

In-Class Exercise 4

- Produce another left-most derivation for the same sentence above.

right-most derivations

- It is also the case that the sentence has two right-most derivations in the grammar:

$$\underline{E} \Rightarrow E + \underline{E}$$

$$\Rightarrow E + E * \underline{E}$$

$$\Rightarrow E + \underline{E} * \text{id}$$

$$\Rightarrow \underline{E} + \text{id} * \text{id}$$

$$\Rightarrow \text{id} + \text{id} * \text{id}$$

In-Class Exercise 5

- Produce another right-most derivation for the same sentence.

In-Class Exercise 6

- Produce the two parse trees for the sentence to demonstrate ambiguity.

- Clearly, we would rather have unambiguous grammars describe our programming language, because ambiguity in the parsing can lead to ambiguity in assigning semantics (meaning) to programs.
- For example, in `id + id * id`, which operation is applied first: addition or multiplication?
- From our math days, we would like the multiplication operator, `*` to be more binding than the addition operator, `+`.

Conditional statements

- As another example, consider the grammar describing conditional statements

$$\begin{aligned} S &::= \text{if } (E) S \\ &\quad | \text{if } (E) S \text{ else } S \\ &\quad | s \\ E &::= e \end{aligned}$$

- Here, the token **e** represents an **arbitrary expression** and the **S** represents a (non-conditional) statement.
- Consider the following sentence form (string of terminals): **if (e) if (e) s else s**
- If we look at this sentence carefully, we see that it nests one conditional statement within another.
- One might ask: To which '**if**' does the '**else**' belong?
- We cannot know; the sentence is ambiguous.

Two left-most derivations for a conditional statement

- More formally, there exist two left-most derivations for this sentence:

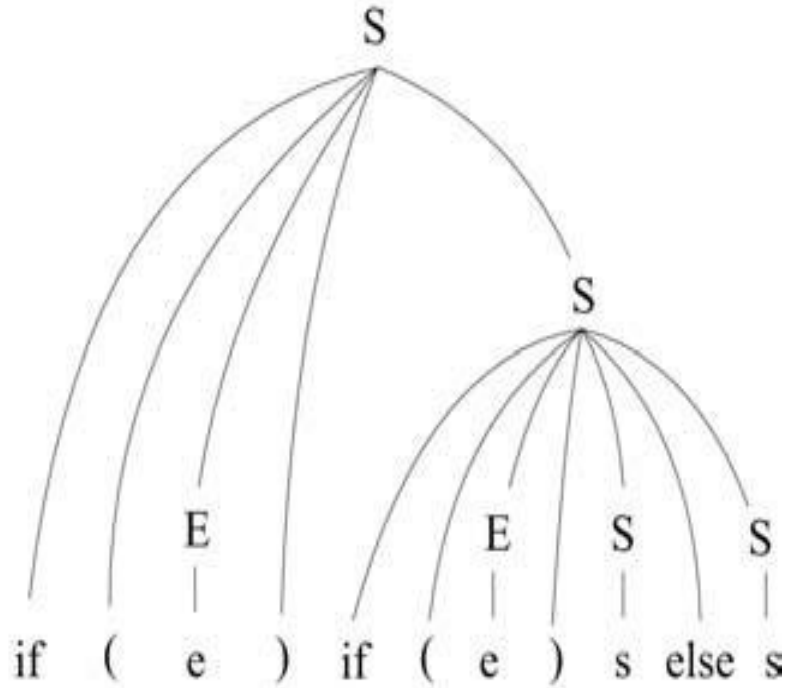
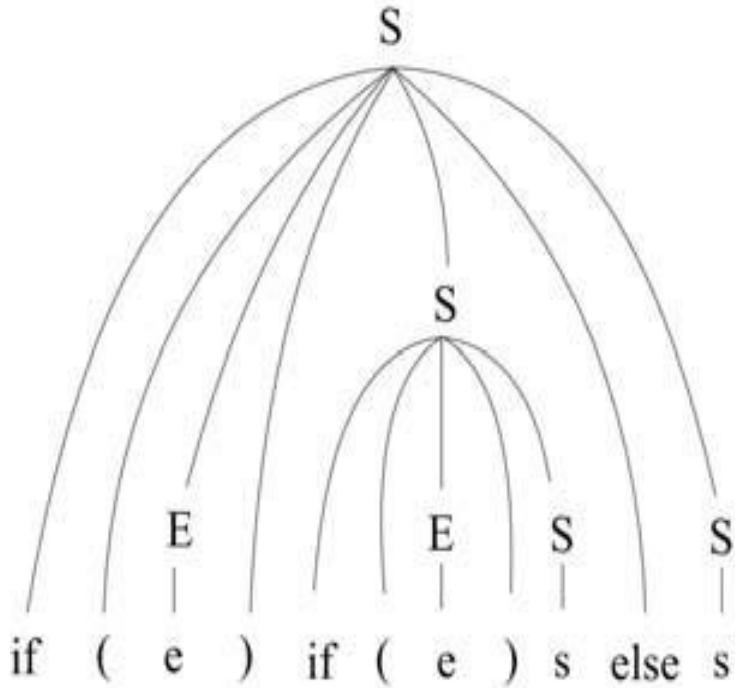
Derivation 1:

$\underline{S} \Rightarrow \text{if } (\underline{E}) S \text{ else } S$
 $\Rightarrow \text{if } (e) \underline{S} \text{ else } S$
 $\Rightarrow \text{if } (e) \text{ if } (\underline{E}) S \text{ else } S$
 $\Rightarrow \text{if } (e) \text{ if } (e) \underline{S} \text{ else } S$
 $\Rightarrow \text{if } (e) \text{ if } (e) s \text{ else } \underline{S}$
 $\Rightarrow \text{if } (e) \text{ if } (e) s \text{ else } s$

Derivation 2:

$\underline{S} \Rightarrow \text{if } (\underline{E}) S$
 $\Rightarrow \text{if } (e) \underline{S}$
 $\Rightarrow \text{if } (e) \text{ if } (\underline{E}) S \text{ else } S$
 $\Rightarrow \text{if } (e) \text{ if } (e) \underline{S} \text{ else } S$
 $\Rightarrow \text{if } (e) \text{ if } (e) s \text{ else } \underline{S}$
 $\Rightarrow \text{if } (e) \text{ if } (e) s \text{ else } s$

Two parse trees for if (e) if (e) s else s



- In general, there is no algorithm that can tell us whether or not an arbitrary grammar is ambiguous.
- That is, ambiguity is not decidable.
- But there are large classes of grammars (and so programming languages) that both can be shown to be decidable and for which efficient parsers can be automatically constructed.
- These classes of grammars are large enough to describe the many programming languages in use today.

- Because of their recursive nature, parsing languages described by context-free grammars requires the use of a **pushdown stack**.
- In general, we can parse any language that is described by a context-free grammar but more often than not, the algorithms that do so involve **backtracking**.
- But there are classes of grammars whose languages may be parsed *without* backtracking; the parsing is said to be **deterministic**.
- At each step of the parsing algorithm, the next step may be determined simply by looking at both the state of the **pushdown stack** and the **incoming symbol** (as the input sentence is scanned from left to right).

Deterministic Parsers categorized

- These deterministic parsers may be roughly divided into two categories:
 - **Top-down parsing:**
 - where the parser begins at the start symbol, S and, step-by-step derives the input sentence and producing either a parse tree or (more likely) an Abstract Syntax Tree (AST) from the root down to its leaves.
 - **Bottom-up parsing:**
 - where the parser begins at the input sentence and, scanning it from left to right, applies the production rules, replacing righthand sides with lefthand sides, in a step-by-step fashion for reducing the sentence to obtain the start symbol.
 - Here the Parse Tree or Abstract Syntax Tree is built from the bottom leaves up to the top root.

Top-Down Deterministic Parsing

- There are two popular top-down deterministic parsing strategies:
 - 1. Parsing by recursive descent and**
 - 2. LL(1) parsing.**
- Both Parsers scan the input from left to right, looking at and scanning just one symbol at a time.
- In both cases, the parser starts with the grammar's start symbol, S as an initial goal in the parsing process;
 - that is, the goal is to scan the input sentence and parse the syntactic entity represented by the start symbol.
- The start symbol is then rewritten, using a BNF rule replacing the symbol with the right-hand side sequence of symbols.

Parsing

- Parsing = *process of determining if a string of tokens can be generated by a grammar*
- For any CF grammar there is a parser that takes at most $O(n^3)$ time to parse a string of n tokens
- Linear algorithms suffice for parsing prog. language source code
- **Top-down parsing** “constructs” a parse tree from the **root** to the **leaves**
- **Bottom-up parsing** “constructs” a parse tree from the **leaves** to the **root**

Predictive Parsing

- ***Recursive descent parsing*** is a top-down parsing method
 - Every nonterminal has one (recursive) procedure responsible for parsing the nonterminal's syntactic category of input tokens
 - When a nonterminal has multiple productions, each production is implemented in a branch of a selection statement based on input look-ahead information
 - It is a common form of top down parsing. It is called recursive as it uses recursive procedures to process the input. Recursive descent parsing suffers from backtracking

Predictive parsing

- **Predictive parsing** is a special form of recursive descent parsing where we use one lookahead token to unambiguously determine the parse operations.
 - That is it has the capability to predict which production is to be used to replace the input string
 - It uses an input buffer, a stack and a parsing table to parse the input and generate a parse tree.

Example Predictive Parser (Grammar)

$type \rightarrow simple$
 | **^ id**
 | **array [simple] of type**
 $simple \rightarrow$ **integer**
 | **char**
 | **num dotdot num**

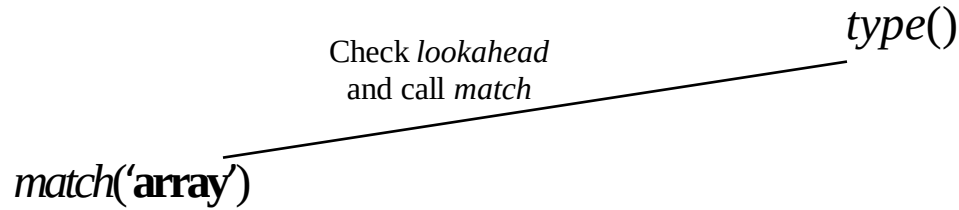
Example Predictive Parser (Program Code)

```
procedure match(t : token);  
begin  
  if lookahead = t then  
    lookahead := nexttoken()  
  else error()  
end;
```

```
procedure type();  
begin  
  if lookahead in { integer, char, num } then  
    simple()  
  else if lookahead = ^ then  
    match('^'); match(id)  
  else if lookahead = array then  
    match('array'); match('['); simple();  
    match(']'); match('of'); type()  
  else error()  
end;
```

```
procedure simple();  
begin  
  if lookahead = integer then  
    match('integer')  
  else if lookahead = char then  
    match('char')  
  else if lookahead = num then  
    match('num');  
    match('dotdot');  
    match('num')  
  else error()  
end;
```

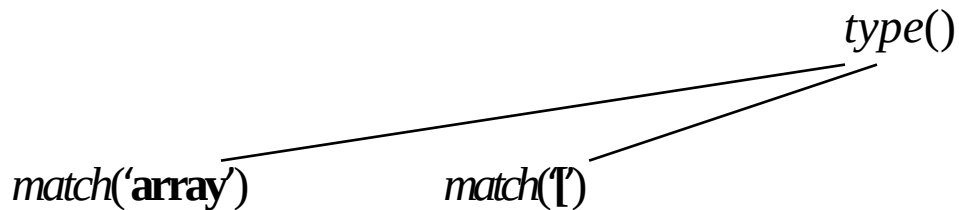
Example Predictive Parser (Execution Step 1)



Input: **array** [**num** **dotdot** **num**] **of** **integer**

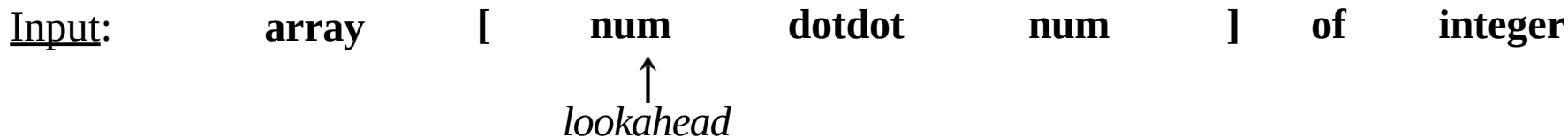
 ↑
 lookahead

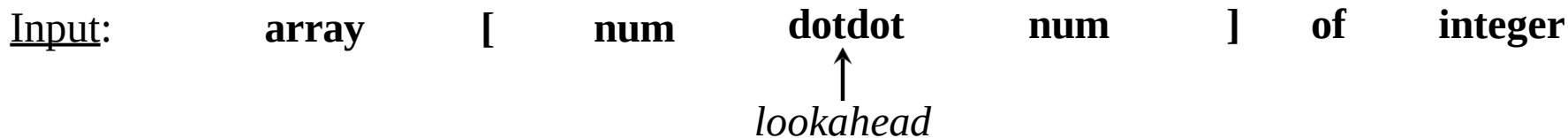
The input string is shown with the word 'array' as the current lookahead. An arrow points from the word 'array' to the word 'lookahead' below it. The words 'array', 'num', 'dotdot', 'num', 'of', and 'integer' are all in bold.



Input: array [num dotdot num] of integer

 ↑
lookahead



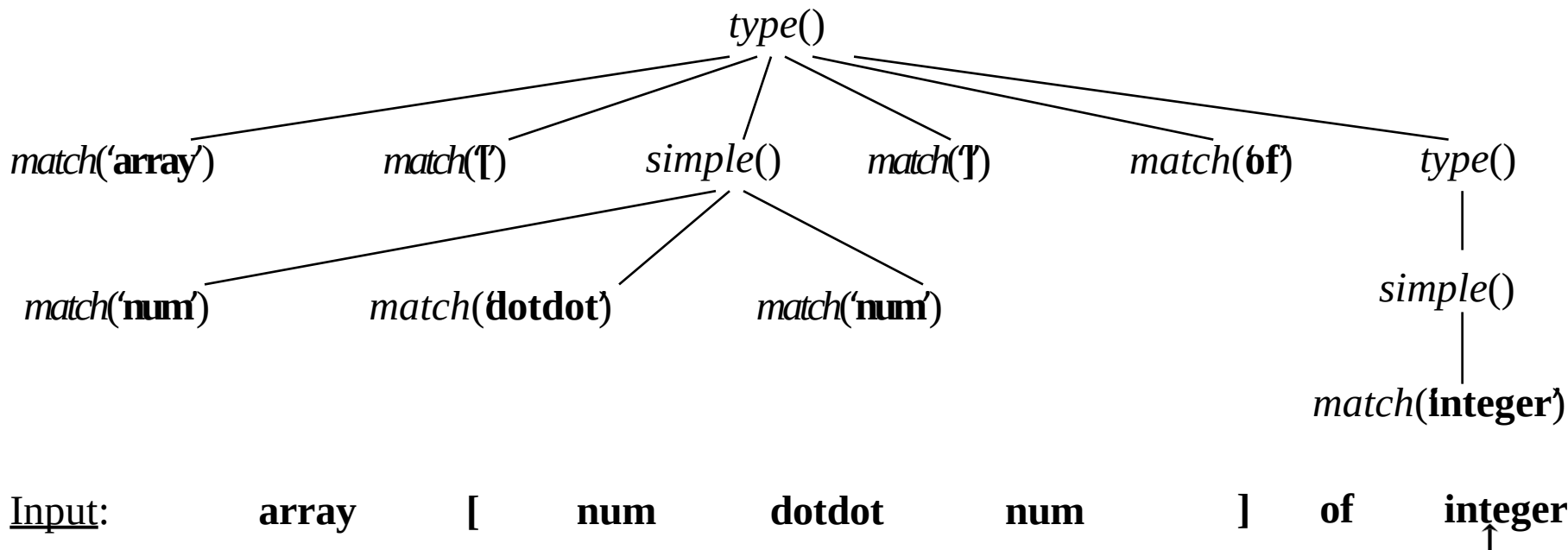








Example Predictive Parser (Execution Step 8)



Homework

1. Consider the grammar:

$$A \rightarrow \mathbf{a} A \mathbf{b} A \mid \mathbf{c}$$

Give a leftmost derivation of the string **aacbcb**.

2. Prove which one of the following strings is recognized by this grammar.

$$S \rightarrow \mathbf{a} S \mathbf{b} \mathbf{b} \mid \varepsilon$$

i. **aabb**

ii. **bb**

iii. **aabbbb**

iv. **aaaabb**

3. Given the following expression grammar, produce a Syntax tree for $2+3*4$.

$$Exp \rightarrow Exp + Exp2$$

$$Exp \rightarrow Exp - Exp2$$

$$Exp \rightarrow Exp2$$

$$Exp2 \rightarrow Exp2 * Exp3$$

$$Exp2 \rightarrow Exp2 / Exp3$$

$$Exp2 \rightarrow Exp3$$

$$Exp3 \rightarrow \mathbf{num}$$

$$Exp3 \rightarrow (Exp)$$

